ADS205 Tutorial 4 - Context-Free Grammars

October 6, 2026

1 ADS205 — Decision, Computation and Language

2 Tutorial 4 — Context-Free Grammars

2.1 Reference card

A **context-free grammar** is a tuple

$$G = (V, \Sigma, R, S),$$

where V is a finite set of variables, Σ is a finite set of terminals, R is a finite set of rules $A \to \alpha$, and $S \in V$ is the start variable.

Notation	Meaning
$uAv \Rightarrow u\alpha v$	one derivation step using $A \to \alpha$
$\Rightarrow^*$	zero or more derivation steps
$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$	the language generated by G
Leftmost derivation	always replace the leftmost remaining variable
Rightmost derivation	always replace the rightmost remaining variable
Ambiguous grammar	some string has two distinct parse trees, equivalently two distinct leftmost derivations

A grammar may be ambiguous even when another unambiguous grammar generates the same language.

---

3 Part A — Reading and constructing CFGs

3.1 Problem 1 — Concepts and counterexamples

Answer each question and justify your answer.

1. Two grammars have the same variables, terminals, and rules but different start variables. Must they generate the same language?
2. If L is context-free and $L' \subseteq L$, must L' be context-free?

3. Can an unambiguous grammar have a leftmost derivation and a different rightmost derivation for the same string?
4. Does every sequence of terminals and variables that can appear during a derivation belong to $L(G)$?

3.2 Problem 2 — Language and derivation

Consider the grammar

$$S \to aSb \mid \varepsilon.$$

1. Give a derivation of `aaabbb`.
2. Describe $L(G)$ precisely.
3. Explain why `aabbb`, `abab`, and `ba` are not generated.
4. Is the grammar ambiguous?

3.3 Problem 3 — A CFG for strings ending in `a`

Give a CFG for

$$L = \{w \in \{a, b\}^* \mid w \text{ ends with } a\}.$$

Show derivations of `a`, `ba`, and `abba`. Explain why ε and `abb` cannot be generated.

3.4 Problem 4 — Matching one symbol with two symbols

Construct a CFG for

$$L = \{0^n 1^{2n} \mid n \geq 0\}.$$

Show a derivation of `001111` and explain why the order of the terminals is correct.

---

4 Part B — Derivations, parse trees, and ambiguity

4.1 Problem 5 — A parse tree and two derivation orders

Let

$$G: \quad S \to 00S1 \mid \varepsilon.$$

1. Draw a parse tree for `000011`.
2. Write its leftmost derivation.
3. Describe $L(G)$.

4.2 Problem 6 — Detect and remove ambiguity

Consider

$$G_1: \quad S \to S + S \mid a,$$

where `+` is a terminal symbol.

1. Give two distinct leftmost derivations of `a+a+a`.
2. What are the two corresponding groupings?
3. Give an unambiguous grammar for the same language.

4.3 Problem 7 — Add exponentiation to an expression grammar

Start from the usual grammar where `*` has higher precedence than `+` and both associate to the left. Add an exponentiation operator `↑` such that

- `↑` has higher precedence than `*`;
- `↑` associates to the right. (e.g., for `a↑b↑c`, we need to compute `b↑c` first, and then `a↑(b↑c)`)

Use your grammar to explain the structures of `a*b↑c` and `a↑b↑c`.

Reference CFG with `+` and `*` where `*` has higher precedence:

$$E \to E + T \mid T,$$
$$T \to T * F \mid F,$$
$$F \to (E) \mid a \mid b.$$

4.4 Optional Tock exploration

The course author's grammar API constructs CFGs from readable production lines. `any_parse` returns one parse tree, while `all_parses` lets us expose ambiguity by enumerating distinct parses of the same word.

[ ]:
```
# Install once in a fresh environment, then import Tock.
# %pip install -q tock
from tock import *

ends_in_a = Grammar.from_lines([
    "S -> a S",
    "S -> b S",
    "S -> a",
])
zero_twice_one = Grammar.from_lines([
    "S -> 0 S 1 1",
    "S -> &",
])

for grammar, samples in [
    (ends_in_a, ["a", "b a", "a b a"]),
```

```
    (zero_twice_one, ["&", "0 1 1", "0 0 1 1 1 1"]),
]:
    display(grammar)
    for word in samples:
        display(any_parse(grammar, word))
```

[ ]:
```
# Tock identifiers use `plus` here to represent the terminal +.
ambiguous_plus = Grammar.from_lines([
    "S -> S plus S",
    "S -> a",
])

word = "a plus a plus a"
parses = list(all_parses(ambiguous_plus, word))
print(f"{word!r} has {len(parses)} parse trees")
for tree in parses:
    display(tree)

assert len(parses) == 2
```

4.5 What to take away

- The start variable is part of a CFG and can change the generated language.
- Recursive rules express dependencies such as matching counts and nesting.
- A parse tree records structural choices. Leftmost and rightmost derivations may list those choices in different orders.
- To prove ambiguity, find one string with two parse trees or two leftmost derivations.
- Ambiguity belongs to a grammar. Inherent ambiguity belongs to a language.
- Expression grammars use separate variables to encode precedence and associativity.